In [1]:
```python
import cv2    # Pragya Singhal , CU24220050, 43
import matplotlib.pyplot as plt
import numpy as np

# 1 & 2. Load image and preprocess (Grayscale + Gaussian Blur)
image = cv2.imread("img.jpeg")
if image is None:
    raise FileNotFoundError("img.jpeg not found in the workspace folder")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# 3. Global Thresholding    # Pragya Singhal , CU24220050, 43
_, thresh_global = cv2.threshold(blurred, 127, 255, cv2.THRESH_BINARY)

# 4. Otsu's Thresholding
_, thresh_otsu = cv2.threshold(
    blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# 5. Adaptive Thresholding    # Pragya Singhal , CU24220050, 43
thresh_adaptive = cv2.adaptiveThreshold(
    blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2
)

# 6. Watershed Segmentation   # Pragya Singhal , CU24220050, 43
# Morphological opening to remove noise
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh_otsu, cv2.MORPH_OPEN, kernel, iterations=2)

# Sure background area
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Finding sure foreground area using distance transform
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.7 * dist_transform.max(), 255, 0)

# Finding unknown region   # Pragya Singhal , CU24220050, 43
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# Marker labelling for watershed
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
watershed_image = image.copy()
markers = cv2.watershed(watershed_image, markers)
watershed_image[markers == -1] = [0, 0, 255]  # Mark boundaries in red

# 7. K-Means Clustering for Color Segmentation
pixel_values = image.reshape((-1, 3))
pixel_values = np.float32(pixel_values)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
k = 3
_, labels, centers = cv2.kmeans(
    pixel_values, k, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS
)
centers = np.uint8(centers)
segmented_data = centers[labels.flatten()]
```

```
kmeans_image = segmented_data.reshape(image.shape)

# 8 & 9. Visualization and Comparison (Steps 8 & 9)
# Pragya Singhal , CU24220050, 43
titles = [
    "Original",
    "Global Threshold",
    "Otsu Threshold",
    "Adaptive Threshold",
    "Watershed",
    "K-Means",
]
images = [
    cv2.cvtColor(image, cv2.COLOR_BGR2RGB),
    thresh_global,
    thresh_otsu,
    thresh_adaptive,
    cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB),
    cv2.cvtColor(kmeans_image, cv2.COLOR_BGR2RGB),
]

plt.figure(figsize=(12, 6))     # Pragya Singhal , CU24220050, 43
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray" if i in [1, 2, 3] else None)
    plt.title(titles[i])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

